



Università di Catania

UNIVERSITÀ DEGLI STUDI DI CATANIA
DIPARTIMENTO DI MATEMATICA E INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA.

Relazione Progetto *Semantic Web*

CyberSecOnto: Automazione dell'Analisi del Rischio Informatico tramite Tecnologie Semantiche

Martina Brancaforte
matricola 1000015074

Prof. Marianna Nicolosi Asmundo

Anno Accademico 2025/2026

Indice dei contenuti

Indice dei contenuti	i
1 Introduzione e Obiettivi	2
1.1 La sicurezza informatica moderna	2
1.2 Obiettivi del progetto	2
2 Modellazione Concettuale	3
2.1 Gerarchia delle Classi	3
2.1.1 Assiomi di Disgiunzione	3
2.2 Proprietà	4
2.2.1 Object Properties	4
2.2.2 Data Properties	4
2.3 Restrizioni Logiche (Description Logics)	5
2.3.1 Restrizione di Cardinalità (Exactly / Min)	5
2.3.2 Restrizione Universale (Only)	5
2.3.3 Restrizione Esistenziale (Some)	5
2.3.4 Operatori Booleani (Intersezione e Negazione)	6
3 Reasoning e Inferenza	7
3.1 Inferenza tramite Classi Equivalenti (OWL)	7
3.1.1 Restrizioni Esistenziali: Sistemi Vulnerabili	7
3.1.2 Datatype Restrictions: Vulnerabilità Critiche	8
3.1.3 Inferenza a Catena: Propagazione dell'Attacco	8
3.1.4 Regola 2: Propagazione del Rischio (High Risk)	8
3.2 Inferenza tramite Classi Equivalenti (OWL)	8
4 Implementazione Software e Automazione	9
4.1 Ambiente di Sviluppo (Visual Studio Code)	9
4.2 Integrazione Python con Owlready2	9
4.3 Script di Generazione Procedurale	10
4.4 Espansione e Popolamento dell'Ontologia	10
5 Interrogazione (SPARQL)	11
5.0.1 Query: Sistemi Critici e relative Vulnerabilità	11
5.0.2 Query: La Catena d'Attacco Completa	11
5.0.3 Query: Dati Sensibili a Rischio (Information Assets)	12
5.0.4 Query: Architettura di Difesa (Mitigazioni)	12
5.0.5 Query: Mappa delle Comunicazioni tra Server	13

	1
5.0.6 Query: Statistiche Vulnerabilità per Severità	13
6 Conclusioni e Sviluppi Futuri	14

1.1 La sicurezza informatica moderna

La sicurezza informatica moderna (Cybersecurity) è caratterizzata da una vasta eterogeneità di informazioni: asset hardware, vulnerabilità software, tipologie di attacco e contromisure. La gestione di queste informazioni tramite database relazionali tradizionali risulta spesso rigida e incapace di evidenziare relazioni implicite, come l'impatto a catena di un guasto o il livello di rischio derivato. In questo progetto è stata sviluppata CyberSecOnto, un'ontologia in linguaggio OWL (Web Ontology Language) progettata per modellare lo scenario della sicurezza aziendale.

1.2 Obiettivi del progetto

L'obiettivo principale non è la semplice archiviazione dei dati, ma l'automazione del processo di analisi del rischio. Attraverso l'uso di tecnologie del Web Semantico, il sistema è in grado di:

- Modellare l'infrastruttura IT e le dipendenze tra i sistemi.
- Inferire automaticamente la criticità delle vulnerabilità basandosi su punteggi standard (CVSS).
- Classificare i sistemi a rischio senza intervento umano, sfruttando sia regole SWRL che inferenze OWL standard.
- Validare la consistenza dei dati tramite restrizioni logiche.

La struttura terminologica (T-Box) è stata definita utilizzando l'editor Protégé. Il dominio è stato modellato attraverso una gerarchia di classi, proprietà semantiche e restrizioni logiche.

2.1 Gerarchia delle Classi

L'albero delle classi è stato notevolmente espanso per riflettere in modo realistico un ecosistema aziendale, raggiungendo un totale di 58 classi. Le entità principali seguono una tassonomia che riflette le componenti di un ecosistema cyber:

- **System:** La classe radice per gli asset hardware. Include le sottoclassi Server, Workstation e dispositivi di rete.
- **InformationAsset:** Un nuovo ramo dedicato alla classificazione dei dati, che comprende asset critici come FinancialData e PII (Personally Identifiable Information).
- **ThreatActor:** Gli agenti di minaccia, ora ulteriormente dettagliati in categorie specifiche come Hacker e NationState.
- **Attack:** Rappresenta le tipologie di attacco, ampliate per includere minacce mirate come Ransomware, ManInTheMiddle e Phishing.
- **Vulnerability:** Rappresenta le debolezze software e hardware (es. CVE).
- **Mitigation:** Rappresenta le contromisure difensive (es. Firewall, Patch), ora posta come classe principale al livello radice della gerarchia.

2.1.1 Assiomi di Disgiunzione

Per garantire la coerenza logica del modello, è stata applicata la disgiunzione (Disjoint Classes) tra le classi di alto livello. Ad esempio, un individuo non può appartenere contemporaneamente alla classe System e alla classe Attack.

2.2 Proprietà

Le relazioni semantiche definiscono le interazioni tra gli individui. Per arricchire l'espressività semantica del modello, sono state definite proprietà logiche avanzate sfruttando le caratteristiche di OWL.

2.2.1 Object Properties

- **hasVulnerability** (Domain: System $\rightarrow$ Range: Vulnerability): Associa un asset alla sua debolezza.
- **exploits** (Domain: Attack $\rightarrow$ Range: Vulnerability): Indica quale falla viene sfruttata da un attacco.
- **mitigates** (Domain: Mitigation $\rightarrow$ Range: Vulnerability): Indica quale difesa risolve il problema.
- **dependsOn** (Domain: System $\rightarrow$ Range: System): Modella la dipendenza operativa tra macchine ed è stata definita come **Transitiva**. Se il Sistema A dipende da B, e B dipende da C, il Reasoner inferisce automaticamente che A dipende da C.
- **communicatesWith** (Domain: System $\rightarrow$ Range: System): Proprietà **Simmetrica** che stabilisce una comunicazione di rete bidirezionale tra due sistemi.
- **storesAsset** (Domain: System $\rightarrow$ Range: InformationAsset): Proprietà **Asimmetrica** che indica la relazione per cui un sistema archivia determinati dati, ma non viceversa.
- **performs** / **isPerformedBy**: Proprietà **Inverse** che permettono di navigare bidirezionalmente l'azione di un ThreatActor che sferra un Attack.

2.2.2 Data Properties

Il dominio informativo è stato ampliato con diverse proprietà di dato:

- **hasCVSSScore** (Domain: Vulnerability $\rightarrow$ Range: xsd:float): Assegna il punteggio numerico di gravità (Common Vulnerability Scoring System). È stato scelto questo tipo di dato per garantire la compatibilità con i calcoli di precisione.
- **hasOS** (Domain: System $\rightarrow$ Range: xsd:string): Mappa il sistema operativo in uso sull'asset.

- **hasPortNumber** (Domain: System $\rightarrow$ Range: xsd:integer): Indica la porta di rete esposta da un sistema.

2.3 Restrizioni Logiche (Description Logics)

Per aumentare la robustezza, l'espressività del modello e garantire il corretto funzionamento delle inferenze, sono state implementate diverse tipologie di restrizioni OWL sfruttando operatori esistenziali, universali, di cardinalità e booleani.

2.3.1 Restrizione di Cardinalità (Exactly / Min)

Per garantire l'integrità dei dati e modellare l'infrastruttura in modo rigoroso, sono stati imposti vincoli numerici:

- **Vulnerability**: È stato imposto che ogni vulnerabilità debba avere un unico punteggio di gravità (`hasCVSSScore exactly 1 xsd:float`). Inoltre, la proprietà `hasCVSSScore` è stata resa Funzionale, generando un'inconsistenza qualora vengano assegnati due punteggi distinti.
- **DatabaseServer**: È stato imposto che abbia esattamente una porta esposta (`hasPortNumber exactly 1 xsd:integer`).
- **Firewall**: Per essere considerato valido, deve proteggere almeno un sistema (`protects min 1 System`).

2.3.2 Restrizione Universale (Only)

Per vincolare in modo esclusivo il raggio d'azione di specifiche entità:

- **Attack**: Un attacco sfrutta soltanto vulnerabilità (`exploits only Vulnerability`). Questo previene errori di modellazione, come il collegamento errato di un attacco direttamente a un server.
- **Firewall**: È stato stabilito che un Firewall mitighi soltanto problemi di rete (`mitigates only NetworkVulnerability`).

2.3.3 Restrizione Esistenziale (Some)

Utilizzata per definire le interazioni indispensabili tra classi:

- **VulnerableSystem**: Classe speciale definita come `System and (hasVulnerability some Vulnerability)` per la classificazione automatica dei sistemi a rischio.

- **Attack:** È stato imposto che un attacco colpisca determinati asset informativi (`targets some InformationAsset`).

2.3.4 Operatori Booleani (Intersezione e Negazione)

Per classificare i sistemi sicuri senza intervento manuale, è stata utilizzata la logica formale combinando l'intersezione logica e la negazione.

- **SecureSystem:** È stata definita tramite classe equivalente come un sistema che non ospita alcuna vulnerabilità: `System and (not (hasVulnerability some Vulnerability))`. Il Reasoner classifica automaticamente gli individui in questa classe quando non presentano falle registrate.

Il valore aggiunto del progetto risiede nella capacità di dedurre nuova conoscenza non esplicitamente asserita, sfruttando le potenzialità del motore inferenziale (Reasoner).

Nella fase iniziale di progettazione, si era valutato l'impiego delle regole SWRL (Semantic Web Rule Language) per gestire i calcoli numerici e le propagazioni condizionali del rischio. Tuttavia, per garantire una maggiore stabilità computazionale, prevenire criticità note dei Reasoner (come HermiT) nella gestione nativa dei confronti matematici esterni e mantenere l'ontologia nel frammento più puro della Logica Descrittiva, si è optato per una soluzione architetturale interamente basata su **Classi Equivalenti (OWL)** e **Datatype Restrictions**.

3.1 Inferenza tramite Classi Equivalenti (OWL)

L'inferenza standard OWL si è dimostrata estremamente potente per classificare le istanze in modo dinamico e automatico, innescando vere e proprie deduzioni a catena all'interno del dominio della sicurezza.

3.1.1 Restrizioni Esistenziali: Sistemi Vulnerabili

Per classificare automaticamente i sistemi in base alla presenza (indipendentemente dalla gravità), sono state utilizzate le restrizioni logiche. La classe **HighRiskSystem** (o **VulnerableSystem**) è stata definita tramite restrizione esistenziale:

`System and (hasVulnerability some Vulnerability)`

Il *Reasoner* analizza gli individui definiti come generici **System**: se rileva la presenza di almeno una relazione **hasVulnerability**, inferisce automaticamente l'appartenenza alla specifica classe di rischio. Similmente, l'intersezione logica e la negazione hanno permesso di inferire automaticamente i sistemi sicuri (**SecureSystem**), ovvero quei sistemi che non ospitano alcuna vulnerabilità registrata.

3.1.2 Datatype Restrictions: Vulnerabilità Critiche

Il superamento delle regole SWRL è avvenuto definendo la classe **CriticalVulnerability** direttamente tramite una restrizione sui tipi di dato (Datatype Restriction). Per classificare le vulnerabilità con un punteggio CVSS critico (≥ 8.0), è stato utilizzato il seguente costrutto logico:

```
Vulnerability and (hasCVSSScore some float[>= 8.0f])
```

L'uso esplicito e rigoroso del tipo di dato `float` si è rivelato fondamentale per risolvere i conflitti logici e garantire che il Reasoner interpretasse correttamente i valori decimali, attivando la classificazione senza incorrere in inconsistenze strutturali.

3.1.3 Inferenza a Catena: Propagazione dell'Attacco

Il vero punto di forza di questa modellazione in puro OWL è la propagazione a catena della conoscenza. Sfruttando la classe **CriticalVulnerability** appena inferita, è stata creata la classe **CriticalAttack**:

```
Attack and (exploits some CriticalVulnerability)
```

Avviando il Reasoner, si genera un effetto a cascata continuo: se una vulnerabilità riceve un punteggio CVSS elevato, entra a far parte di **CriticalVulnerability**; di conseguenza, qualsiasi attacco che la sfrutta viene automaticamente promosso e inferito come **CriticalAttack**. Questo dimostra la solidità del modello nel dedurre scenari complessi di propagazione delle minacce senza la necessità di aggiornare manualmente le relazioni.

3.1.4 Regola 2: Propagazione del Rischio (High Risk)

Questa regola propaga la criticità dalla vulnerabilità al sistema che la ospita.

3.2 Inferenza tramite Classi Equivalenti (OWL)

Per rispondere alla necessità di classificare i sistemi in base alla presenza di vulnerabilità (indipendentemente dalla gravità), è stata utilizzata una Classe Equivalente definita tramite restrizione esistenziale. La classe **VulnerableSystem** è stata definita come:

```
System and (hasVulnerability some Vulnerability)
```

4 Implementazione Software e Automazione

Per gestire la complessità e garantire la scalabilità del progetto, lo sviluppo non si è limitato all'uso dell'editor grafico Protégé. È stata implementata una pipeline di automazione completa per simulare scenari Enterprise complessi e popolare dinamicamente la base di conoscenza, garantendo al contempo la massima compatibilità e stabilità con i Reasoner standard.

4.1 Ambiente di Sviluppo (Visual Studio Code)

L'intero ciclo di vita del software è stato gestito utilizzando Visual Studio Code. Il progetto è stato strutturato in un repository GitHub contenente:

- I file sorgente dell'ontologia, nello specifico lo schema architetturale di base (*Untitled.rdf*) e l'ontologia finale istanziata (*cyberseconto_populated.owl*).
- Gli script Python di automazione, focalizzati sulla generazione procedurale (*popolamento.py*).
- Gli script di test e validazione (*test_sparql.py*, *debug_onto.py*).

L'uso di un sistema di controllo versione ha permesso di tracciare l'evoluzione del modello semantico e di separare in modo netto la logica applicativa dai dati ontologici.

4.2 Integrazione Python con Owlready2

L'interazione programmatica con l'ontologia è stata realizzata tramite la libreria Owlready2, che consente di mappare le classi OWL in classi Python. Questo ha permesso di manipolare individui e proprietà come oggetti nativi del linguaggio, facilitando enormemente l'automazione dei processi.

4.3 Script di Generazione Procedurale

Lo script `popolamento.py` funge da simulatore di infrastruttura, invece di popolare l'ontologia manualmente, lo script esegue le seguenti operazioni fondamentali:

- Carica la T-Box di base (*Untitled.rdf*) contenente tutte le 58 classi e le relative restrizioni logiche.
- Genera dinamicamente le istanze del dominio (server, database, vulnerabilità e attori delle minacce).
- Assegna rigorosamente i tipi di dato necessari per garantire il corretto funzionamento del Reasoner. Ad esempio, impone l'uso esplicito del tipo `float` per i punteggi CVSS, prevenendo i noti problemi di parsing matematico di motori inferenziali come HermiT.
- Esporta l'infrastruttura completa nel file *cyberseconto_populated.owl*, rendendolo immediatamente pronto per l'analisi e le inferenze native all'interno di Protégé.

4.4 Espansione e Popolamento dell'Ontologia

Nella sua versione finale, l'architettura dell'ontologia è stata estesa fino a comprendere 58 classi. Il popolamento procedurale ha permesso di modellare uno scenario aziendale realistico e denso di interazioni. Nello specifico, sono stati istanziati diversi attori delle minacce (come `Hacker_Anonymous` e `Cyber_Mafia_Group`), vulnerabilità critiche associate a punteggi CVSS reali, e un'articolata rete di difese, tra cui firewall perimetrali e sistemi di cifratura AES256. Questo ecosistema permette di mappare con precisione gli asset informativi, come dati personali (PII) e finanziari, ai server che li ospitano, demandando interamente al Reasoner OWL il compito logico di dedurre le catene di propagazione del rischio.

Le query SPARQL presentate in questo capitolo sono state il frutto di un processo iterativo di progettazione. La suite di interrogazioni è stata notevolmente ampliata per mappare scenari complessi e testata programmaticamente tramite script Python sfruttando la libreria Owlready2. Di seguito vengono presentate le query fondamentali per l'analisi automatizzata del rischio.

5.0.1 Query: Sistemi Critici e relative Vulnerabilità

Questa query estrae tutti i sistemi compromessi da vulnerabilità con un punteggio CVSS superiore a 7.0, ordinandoli in modo decrescente per gravità.

```
PREFIX cyber: <http://www.semanticweb.org/martibra/ontologies/cybersOnto#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
```

```
SELECT ?system ?vuln ?cvss
WHERE {
    ?system cyber:hasVulnerability ?vuln .
    ?vuln cyber:hasCVSSScore ?cvss .
    FILTER (?cvss > 7.0)
}
ORDER BY DESC(?cvss)
```

5.0.2 Query: La Catena d'Attacco Completa

L'interrogazione ricostruisce l'intera catena di attacco, mappando in un'unica vista gli attori delle minacce, la tipologia di attacco eseguito (es. Ransomware, DDoS), le vulnerabilità sfruttate e gli specifici asset informativi presi di mira.

```
PREFIX cyber: <http://www.semanticweb.org/martibra/ontologies/cybersOnto#>
```

```

SELECT ?actor ?attack ?asset ?vuln
WHERE {
    ?actor cyber:performs ?attack .
    ?attack cyber:targets ?asset .
    ?attack cyber:exploits ?vuln .
}

```

5.0.3 Query: Dati Sensibili a Rischio (Information Assets)

Questa estrazione identifica quali asset informativi aziendali (come dati finanziari dei clienti o dati dei dipendenti) si trovano salvati su server che presentano vulnerabilità critiche (CVSS ≥ 8.0).

```
PREFIX cyber: <http://www.semanticweb.org/martibra/ontologies/cybersOnto#>
```

```

SELECT DISTINCT ?asset ?system ?vuln ?cvss
WHERE {
    ?system cyber:storesAsset ?asset .
    ?system cyber:hasVulnerability ?vuln .
    ?vuln cyber:hasCVSSScore ?cvss .
    FILTER (?cvss >= 8.0)
}
ORDER BY DESC(?cvss)

```

5.0.4 Query: Architettura di Difesa (Mitigazioni)

L'interrogazione mappa le difese attive nell'infrastruttura, mostrando esplicitamente quale mitigazione (come un Firewall) protegge un determinato sistema e quale vulnerabilità va a neutralizzare.

```
PREFIX cyber: <http://www.semanticweb.org/martibra/ontologies/cybersOnto#>
```

```

SELECT ?mitigation ?system ?vuln
WHERE {
    ?mitigation cyber:protects ?system .
    ?mitigation cyber:mitigates ?vuln .
}

```

5.0.5 Query: Mappa delle Comunicazioni tra Server

Sfruttando la proprietà logica simmetrica `communicatesWith`, la query estrae la mappa topologica delle comunicazioni interne tra i server dell'infrastruttura.

```
PREFIX cyber: <http://www.semanticweb.org/martibra/ontologies/cybersOnto#>
```

```
SELECT ?serverA ?serverB
WHERE {
    ?serverA cyber:communicatesWith ?serverB .
}
```

5.0.6 Query: Statistiche Vulnerabilità per Severità

Quest'ultima interrogazione aggrega i dati dell'ontologia per fornire un conteggio statistico delle vulnerabilità presenti, suddividendole nelle tre fasce di rischio standard: Critico (9.0-10.0), Alto (7.0-8.9) e Medio (4.0-6.9).

```
PREFIX cyber: <http://www.semanticweb.org/martibra/ontologies/cybersOnto#>
```

```
SELECT
    (COUNT(IF(?cvss >= 9.0, 1, ?unbound)) AS ?critical)
    (COUNT(IF(?cvss >= 7.0 && ?cvss < 9.0, 1, ?unbound)) AS ?high)
    (COUNT(IF(?cvss >= 4.0 && ?cvss < 7.0, 1, ?unbound)) AS ?medium)
WHERE {
    ?v cyber:hasCVSSScore ?cvss .
}
```

Il progetto CyberSecOnto ha ampiamente dimostrato l'efficacia e il valore aggiunto delle tecnologie del Web Semantico nel dominio della sicurezza informatica. Rispetto ai tradizionali database relazionali, caratterizzati da una forte rigidità strutturale, l'approccio basato sui grafi semantici ha permesso di mappare ed estrarre automaticamente relazioni implicite e complesse. L'uso di proprietà logiche avanzate (come quelle transitive, simmetriche e inverse) si è rivelato fondamentale per l'analisi proattiva dell'impatto a catena all'interno di infrastrutture IT complesse.

L'architettura implementata, ora estesa a 58 classi, ha evidenziato una perfetta complementarità tra i diversi strumenti tecnologici adottati, valorizzando la pura Logica Descrittiva:

- **OWL e Datatype Restrictions:** lo standard OWL ha garantito la coerenza della T-Box, la validazione dei dati e la classificazione automatica concettuale. In particolare, l'utilizzo nativo delle *Datatype Restrictions* per i confronti matematici (come la valutazione della gravità del CVSS) ha permesso di superare le ben note instabilità computazionali delle regole esterne SWRL, garantendo un'inferenza a catena fluida, robusta e interamente gestita dal Reasoner.
- **Python ed Owlready2:** hanno permesso di scalare il progetto automatizzando il popolamento procedurale di uno scenario Enterprise realistico, iniettando istanze rigorosamente tipizzate e garantendo la perfetta interazione dei dati con le rigide regole dell'ontologia.

La validazione finale, integrata da un'estesa suite di query SPARQL, ha confermato la correttezza delle deduzioni logiche. Il sistema ha dimostrato di poter generare autonomamente nuova conoscenza, dall'estrazione dell'intera catena d'attacco fino alla classificazione autonoma degli attacchi critici, identificando gli asset prioritari senza alcun intervento manuale.

Come **sviluppi futuri**, il framework CyberSecOnto potrebbe essere esteso per integrare dinamicamente feed di minacce in tempo reale (come il database MITRE ATT&CK o i cataloghi CISA KEV) o per interfacciarsi con sistemi di monitoraggio aziendali (SIEM). Questo permetterebbe di automatizzare non solo la scoperta del rischio, ma anche la raccomandazione contestuale delle contromisure (Mitigation) più idonee.

I sorgenti completi del progetto, l'ontologia di base, la versione popolata e gli script di automazione sono liberamente consultabili e riproducibili presso la repository GitHub dedicata: <https://github.com/Marty14/Semantic-Web>.